Decision index

Every decision, oldest first. Read one section on demand with `doc_get_section`; do not read `docs/decision-records.typ` end to end.

Decision	Title
D1	Ownership lifetime
D2	Dictionary encoding (decided, applied)
D3	Value typing and `nor`'s universe
D4	Program classification (Wave 2)
D5	Property-test framework (W1)
D6	Reproducibility of the verification suite
D7	The refinement loop (demonstration)
D8	Parser generalization (W2)
D9	The loop executes the compiler's output (applied)
D10	The verifier is an SMT solver in-process (Z3), not a proof assistant
D11	Fixed-spec mode: the loop repairs implementations, not specs
D12	Set-theoretic typing: structural and nominal as one record
D13	Decision models: a symbolic table, optional, proven rather than generated
D14	Sorts are a pair, and the pair is landed (W3)
D15	Unification is first-order, image-resident, and nominal (W4)
D16	Metaobject encoding: a tagged tensor, address identity, and explicit levels
D17	Metaobject Protocol scope: open implementation, fixed meta-level, kernel boundary
D18	Scale Basic English: a literal controlled grammar (W2.1)
D19	Multiple dispatch: image-resident generic functions, type-first selection, static fast path (W10)
D20	Channels: image-resident communication endpoints (W6)
D21	Word Metaobjects: image-resident word definitions (D16 Stage 2)
D22	Scheduler: per-process cooperative multitasking (W5)
D23	Verify: a narrow exact-text layer for the emitter artifact (W1)
D24	Executable numeric `has` conditions
D25	The verification type lattice: one coordinate system for Z3, Hedgehog, and the refiner
D26	ChannelEvents (W7)
D27	Review hardening: the trust boundaries hold under adversarial input
D28	Thin follow-ons: the fixed-spec CLI and the no-condition subject
D29	The event-stream as the program (W8)
D30	Linear ownership (W9)
D31	Predicate sorts (D16/D17 stage 3)
D32	Classification: one membership verdict, `class-of`, entry kinds
D33	Existential `some` (D18 stage 1)

Decision	Title
D34	The filtering word for comprehensions
D35	Guard metaobjects (D16/D17 stage 4, guard half)
D36	ValueSet constructors: named, image-resident narrowing (D16/D17 stage 3, constructor half)
D37	Value sets, field types, and set roots: naming the thing, not the mechanism
D38	Identity and ChannelEvents: full words, no borrowed acronym
D39	Set-operation protocol methods: the operation is an image method
D40	Speech-first names: Scale is the product, Lia is the mascot
D41	The guard-lattice bridge: a document refinement can name an image-resident guard
D42	DSPy-shaped model-backed programs are an optional, prepared track
D43	The full cell system: classes, slots, instances, precedence, combination, closures, metaclasses
D44	Cell-system encoding: tags, codecs, roots, persistence
D45	Closures and lexical scope
D46	Classes, slots, and instances
D47	Class precedence and nominal class dispatch
D48	Method qualifiers and combination
D49	Metaclasses and the cell-class closure
D50	Conditions and restarts
D51	Control refinement
D52	Reference linearity and reclaim candidates
D53	Opt-in compaction with stable addresses
D54	Static ownership contracts
D55	Transition emission (the D18 conditional, now that event semantics exist)
D56	Witness-guided candidate generation for symbolic domains
D57	Violation reporting, phase P0
D58	Disjunction (the D18 `or` stage)
D59	Higher-order unification (Miller patterns)
D60	Tensor algebra beyond scalars (W11)
D61	Violation reporting, phase P1 (semantic stages)
D62	Violation reporting, phase P2 (SMT models and unsat cores)
D63	Preemptive supervision and reduction deadlines (W5)
D64	Actor handles are private capabilities
D65	Actor mailboxes, self, and links
D66	Trapped exits
D67	Monitors and the named registry
D68	Closing the actor boundaries
D69	One actor-word guard for quotation runs and set literals

Decision	Title
D70	Discourse rollback (P3)
D71	Minimum-edit repair search (P4)
D72	Presentation layer (P5)
D73	Promoting decision tables (D13)
D74	The Basic English table surface and the diagnostic sink
D75	Reserved actor system messages
D76	Character offsets for source violations
D77	A syntactic discourse contradiction detector
D78	The SMT violation in the showcase sink
D79	Typed proposals, advice, and the frozen specification
D80	Memory vocabulary, handles, views, and actor epochs
D81	Usage modes and trace linear types
D82	Reclamation roots include every live actor
D83	Epoch allocation ranges and promotion
D84	Reflection.Emit is a JIT-only compiler backend
D85	Epoch allocation recording in the scheduler
D86	Promotion at send boundaries
D87	Host-level persistence filters by epoch
D88	Physical reclamation of closed epochs
D89	TensorHandle, TensorView, and the managed arena contract
D90	Memory representation freeze
D91	Real and Natural sorts in the SMT encoding
D92	Sort-faithful F\# emission
D93	The unified language is the programmable IR beneath Basic English
D94	Evaluation errors are one taxonomy with typed fault families
D95	The kernel is the abstract machine behind one C ABI
D96	Pattern matching starts with the runtime IR and the ground matcher
D97	One general IR carries the portable abstractions
D98	The portable IR and high-level slices leave the kernel
D99	The kernel owns the execution core
D100	The kernel loads and validates the Prelude image
D101	The kernel executes an exported image unit
D102	Boundary proof for the kernel machine
D103	Primitive operands and results cross the ABI as tensors
D104	Pattern templates elaborate to closed patterns
D105	The ABI imports and exports the retained image
D106	Callbacks can be replaced on a live handle
D107	DDD is adopted with Typst and current ABI names
D108	The ABI forwards named host calls

Decision	Title
D109	The ABI forwards transport payloads
D110	Image units may recurse within a call-depth bound
D111	Core stack and arithmetic opcodes live in the VM
D112	Association-vector primitives live in the kernel store
D113	Image instructions expose association roots
D114	Duplicated granular cases leave; portable JSON is shared
D115	Dynamic word definitions run as lowered images
D116	Initial port scope and non-porting tracks
D117	Closures capture lexical slots
D118	Tensor and text inspection primitives
D119	Expression lowering is total over the closed node universe
D120	Compiled artifacts exclude history, bookmarks, and rollback
D121	Lowered let reuses the interpreter's let word; lambda stays declined
D122	Lexical bindings resolve through the frame chain and lowered slots
D123	Lambda parameters bind to slots in a named unit
D124	A differential corpus guards the lowered VM against the interpreter
D125	Pattern matching compiles to inspection, branches, and slots
D126	The IL writer is a JIT-only backend behind a conditional compile
D127	Port review after D126
D128	The writer targets a runtime harness over the real machine state
D129	The language specification is canonical
D130	Alpha decisions: reload, writer units, interpreter-only nodes, C adapter
D131	Surface, subtyping, Basic English, and verification decisions
D132	Sequences and recursive types are target; multisets and co-products wait
D133	Prelude deferral, free-variable capture, distribution, ratchets
D134	The writer emits one method per unit and compiles direct lambda calls
D135	Closed-decision ratchets
D136	The C adapter and the conformance harness
D137	The repository is a Polylith workspace
D138	The module dependency map and the corrected component cut
D139	Verification, demo, CLI, and development split
D140	The Nu node, the host skeleton, and the jj pilot
D141	The demo resolves its content root in a checkout and in a payload
D142	The prose surface: sentence capitalization, no comma, `.prose` files
D143	The comma delimits the multiple-choice construct
D144	Spaced names, text with spaces, nested matches, and the program result
D145	The prose program is a sentence sequence; no kebab-case for now
D146	The prose program runs on the kernel stack
D147	Prose as a script-only shell

Decision	Title
D148	The first test batch: lowering differential, prose properties, lowered-IR snapshot
D149	The `.prose` file reader and CLI mode
D150	`LoweredVm` becomes `LoweredAbstraction`
D151	Drop the unified Julia/Elixir language; integrate instances by actor IPC
D152	The actor wire protocol is implemented in-repo, Cap'n Proto-inspired
D153	Protobuf-style schema tooling over a Cap'n Proto-compatible wire
D154	The first actor IPC slice: a numeric envelope over a Unix socket
D155	S2: worker capabilities and remote supervision
D156	The BEAM wire harness: an Elixir peer and listener
D157	Language-level remote send records a transport request
D158	Remote replies are delivered into the sender's mailbox
D159	The wire golden vectors and reference interop
D160	The live program is a typed term list
D161	P1: the core term shape and its portable document
D162	P2: the kernel structural verifier and receipt
D163	P3a: the token-to-term boundary
D164	P3b: the typed control state
D165	P3b implemented: the typed control state
D166	P3c: quotation code values and structural history
D167	P3d: remove the text side channels
D168	The F-sharp-equivalent pipeline
D169	Curried pipeline stages
D170	Explicit partial application with holes
D171	The reverse pipe
D172	Function values, currying, and recursion classification
D173	P4a first-order type constraint engine
D174	One built-in word index, checked against the dispatcher
D175	The ABI is reset to 1 with independent handles
D176	The test suite is a separate JIT-only deployable
D177	Prose logic bindings and bound closure calls
D178	Event history is opt-in and uses a persistent step chain
D179	Split the evaluator state from the scheduler (staged)
D180	Kernel changes require a NativeAOT build and C conformance
D181	Nu integration and Gaia-on-ImSim plan
D182	Isolate Scale-only work in a sparse jj workspace
D183	Host words and the first shell effects
D184	Books and spells are packages
D185	Abstractions, logic, and type retrieval

Decision	Title
D186	Basic English is the historical name for the Prose sentence grammar
D187	The `with` connective
D188	Book call-order validation
D189	The strict callable contract in the lowered path
D190	The lowered instruction set is `LoweredInstructions`
D191	One execution-root registry
D192	Image and package schema 2 carry unit groups
D193	Group call frames and the call/closure split
D194	Automatic book recursion and optional markers
D195	Logic and cycle definitions
D196	F\# abstraction and manifest classification
D197	Classification metadata wire
D198	Book package contract
D199	Open a book